

# Exploring Battle Dynamics Through Lanchester's Differential Equations

## Limitations and Model Improvements

Tanish GHOSH

BSD-BG-2420

Indian Statistical Institute, Bangalore Center

bsdbg2420@isibang.ac.in

26 APR 2026

### Abstract

This project studies Lanchester's combat model, beginning with the classical two-team ordinary differential equation (ODE) formulation and then extending it to a two-dimensional spatial setting. The basic model captures attrition between two opposing forces, but it is limited because it ignores geography, movement, reinforcements, and local structure. To address these limitations, the battlefield is discretized into a finite grid of cells, and each cell is assigned a pair of coupled ODEs for the two teams. This converts the spatial problem into a large system of ODEs that can be simulated numerically in R. Several scenarios are explored, including opposite-boundary attacks, opposite-corner engagement, attacker-defender interaction, and mutual defense. The project also discusses how the classical square-law idea can hold locally in a cell or approximately globally under strong mixing, while noting that exact analytical winning conditions are generally lost once movement and spatial coupling are added.

## Contents

|                                                                           |          |
|---------------------------------------------------------------------------|----------|
| <b>1 Introduction: The Classical Lanchester Combat Model</b>              | <b>3</b> |
| 1.1 Numerical visualization of the basic model in R . . . . .             | 4        |
| <b>2 Why the Classical Model Needs Improvement</b>                        | <b>4</b> |
| <b>3 Extension to a 2D Spatial Combat Model</b>                           | <b>5</b> |
| 3.1 Idea of the discretization . . . . .                                  | 5        |
| 3.2 Movement rule . . . . .                                               | 5        |
| 3.3 Local square law . . . . .                                            | 5        |
| <b>4 R Implementation of the 2D Spatial Model</b>                         | <b>6</b> |
| 4.1 Scenario I: Opposite Boundaries Attacking Toward the Middle . . . . . | 6        |
| 4.2 Scenario II: Opposite Corners . . . . .                               | 7        |
| 4.3 Scenario III: One Attacker and One Defender . . . . .                 | 7        |
| 4.4 Scenario IV: Both Teams Defending . . . . .                           | 8        |

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>5 Winning Conditions</b>                     | <b>9</b>  |
| 5.1 Local winning condition . . . . .           | 9         |
| 5.2 Approximate global interpretation . . . . . | 10        |
| 5.3 Steady-state viewpoint . . . . .            | 10        |
| <b>6 Limitations and Further Directions</b>     | <b>10</b> |
| <b>7 Conclusion</b>                             | <b>11</b> |
| <b>References</b>                               | <b>11</b> |
| <b>Appendices</b>                               | <b>12</b> |
| Annexure: All R Code . . . . .                  | 12        |

# 1 Introduction: The Classical Lanchester Combat Model

The classical Lanchester square-law model describes the time evolution of two opposing forces, say  $A(t)$  and  $B(t)$ , where the rate at which one force loses units is proportional to the size of the opposing force.

A standard form is

$$\begin{aligned}dA/dt &= -\beta B, \\dB/dt &= -\alpha A,\end{aligned}$$

where  $\alpha$  and  $\beta$  are positive constants measuring combat effectiveness.

This model is attractive because it is simple and mathematically tractable. It also leads to a useful invariant. Dividing the first equation by the second gives

$$dB/dA = \alpha A/\beta B$$

Rearranging and integrating,

$$\alpha A dA = \beta B dB$$

so,

$$\alpha A^2 - \beta B^2 = C$$

where  $C$  is constant along trajectories.

Therefore, if the initial values are  $A(0) = A_0$  and  $B(0) = B_0$ , then

$$\alpha A^2(t) - \beta B^2(t) = \alpha A_0^2 - \beta B_0^2$$

If Team  $B$  is completely eliminated first, then  $B(t) = 0$  at the end and Team  $A$  survives with

$$A_{final}^2 = A_0^2 - \frac{\beta}{\alpha} B_0^2$$

Hence a simple winning condition for Team  $A$  is

$$\alpha A_0^2 > \beta B_0^2$$

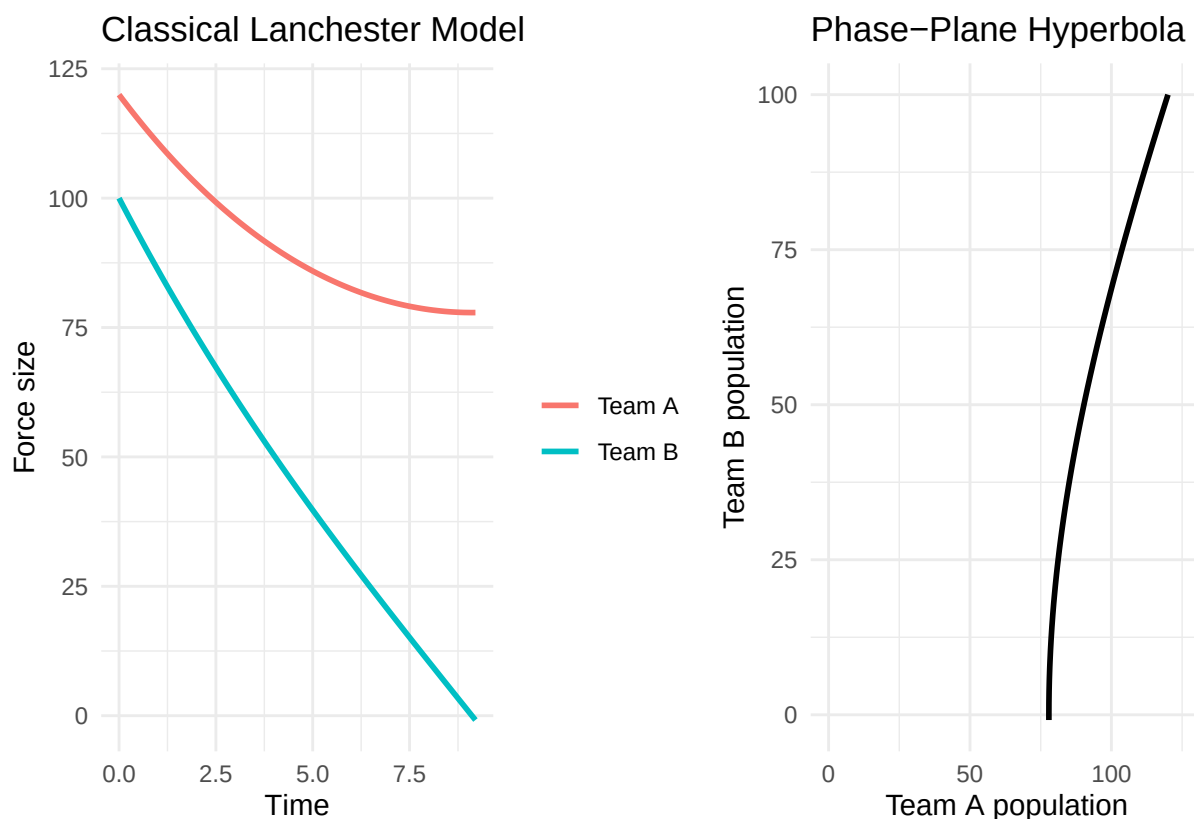
Similarly, Team  $B$  wins when

$$\beta B_0^2 > \alpha A_0^2$$

This is the classical square-law conclusion.

## 1.1 Numerical visualization of the basic model in R

The plots below shows the gradual attrition of both forces. The side with the larger effective initial strength tends to survive.



The graph of *Team B* vs. *Team A* shows the idea of Lanchester's square law of collapse, where when the forces are large initially, they decrease at a linear rate with respect to each other. However, when they get smaller, the quadratic rate kicks in and one of the team collapses.

## 2 Why the Classical Model Needs Improvement

Although elegant, the classical model has important limitations:

- it treats both forces as spatially mixed and ignores location,
- it assumes constant effectiveness,
- it ignores reinforcements,
- it does not distinguish between offensive and defensive movement,
- it is deterministic and therefore does not reflect randomness,
- it assumes all units are homogeneous.

These assumptions are too restrictive for many realistic settings. A more flexible model should allow movement across space and should permit local encounters between the two teams.

### 3 Extension to a 2D Spatial Combat Model

If space were treated continuously, one would generally need partial differential equations (PDEs) to formulate this model. However, to simplify it and use ODEs, discretization is often a better choice because it keeps the model computationally manageable.

To include spatial effects, let the battlefield be a two-dimensional region divided into a finite grid. For a grid of size  $n \times n$ , define

$$A_{ij}(t), B_{ij}(t), i, j = 1, 2, \dots, n$$

where  $A_{ij}(t)$  and  $B_{ij}(t)$  represent the troop counts of the two teams in cell  $(i, j)$  at time  $t$ .

A general spatial combat model can be written as

$$\begin{aligned}\frac{dA_{ij}}{dt} &= -\beta B_{ij} + Inflow_{ij}^A - Outflow_{ij}^A + R_{ij}^A(t), \\ \frac{dB_{ij}}{dt} &= -\alpha A_{ij} + Inflow_{ij}^B - Outflow_{ij}^B + R_{ij}^B(t).\end{aligned}$$

Here:

the terms  $-\beta B_{ij}$  and  $-\alpha A_{ij}$  represent local attrition inside the cell, the inflow and outflow terms represent troop movement between adjacent cells,  $R_{ij}^A(t)$  and  $R_{ij}^B(t)$  represent reinforcements or external sources.

We now convert the battlefield into a finite system of ODEs.

#### 3.1 Idea of the discretization

Suppose the battlefield is divided into a  $4 \times 4$  grid. Then there are 16 cells. Since each cell contains two state variables, the full system has 32 ODEs.

Instead of solving a PDE, we let each cell communicate with its neighboring cells through flow terms. This gives a compartment-style spatial model.

#### 3.2 Movement rule

For a cell, movement is handled by:

- inflow from neighboring cells,
- outflow to a preferred neighboring cell,
- no movement outside the boundary.

This is simple enough to implement numerically, but still captures the idea of marching, flanking, retreating, and concentration of force.

#### 3.3 Local square law

If movement is temporarily ignored in one cell, then that cell behaves like the classical Lanchester model:

$$\frac{dA_{ij}}{dt} = -\beta B_{ij},$$

$$\frac{dB_{ij}}{dt} = -\alpha A_{ij}.$$

Hence the local invariant is

$$\alpha A_{ij}^2 - \beta B_{ij}^2 = C_{ij}.$$

This means the square law still holds locally.

When movement is included, this exact invariant is generally lost globally. Still, the square-law idea can remain a useful approximation in the main contact zone.

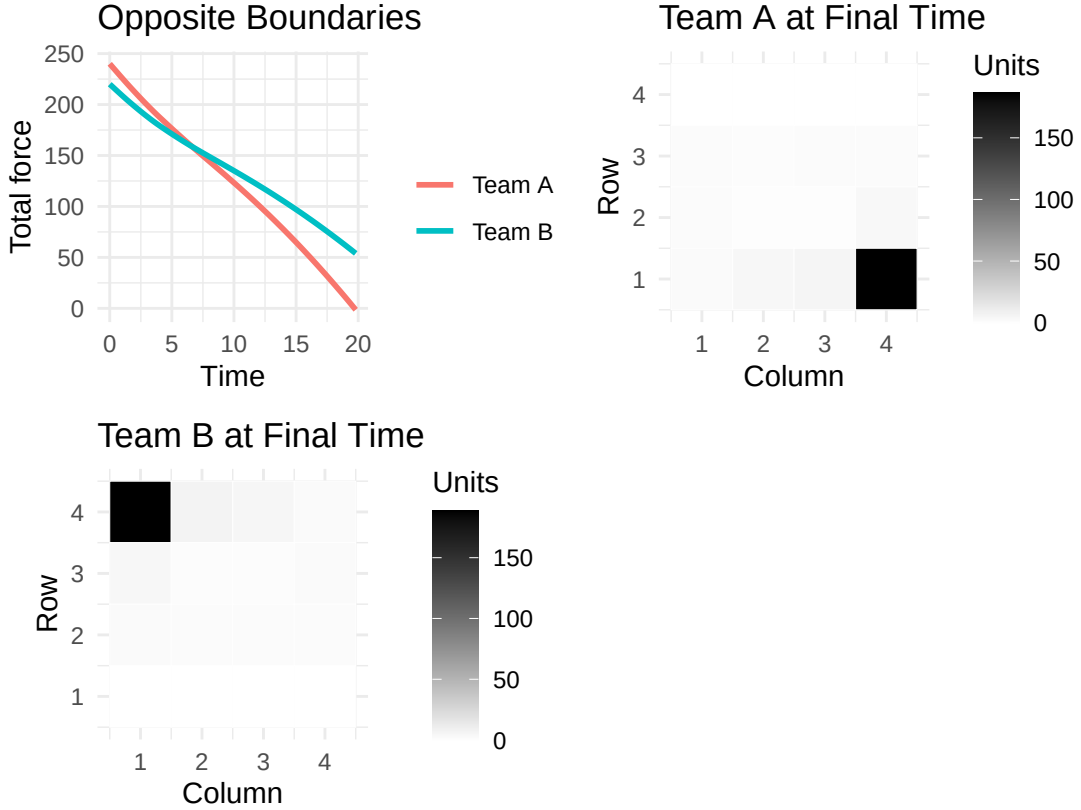
## 4 R Implementation of the 2D Spatial Model

The code (*given in the appendix*) implements a discrete 2D spatial Lanchester's model on an  $n \times n$  grid using only `deSolve` and `ggplot2`.

### 4.1 Scenario I: Opposite Boundaries Attacking Toward the Middle

In this case, Team A enters from the left boundary and moves east, while Team B enters from the right boundary and moves west. The two teams meet in the middle region of the grid.

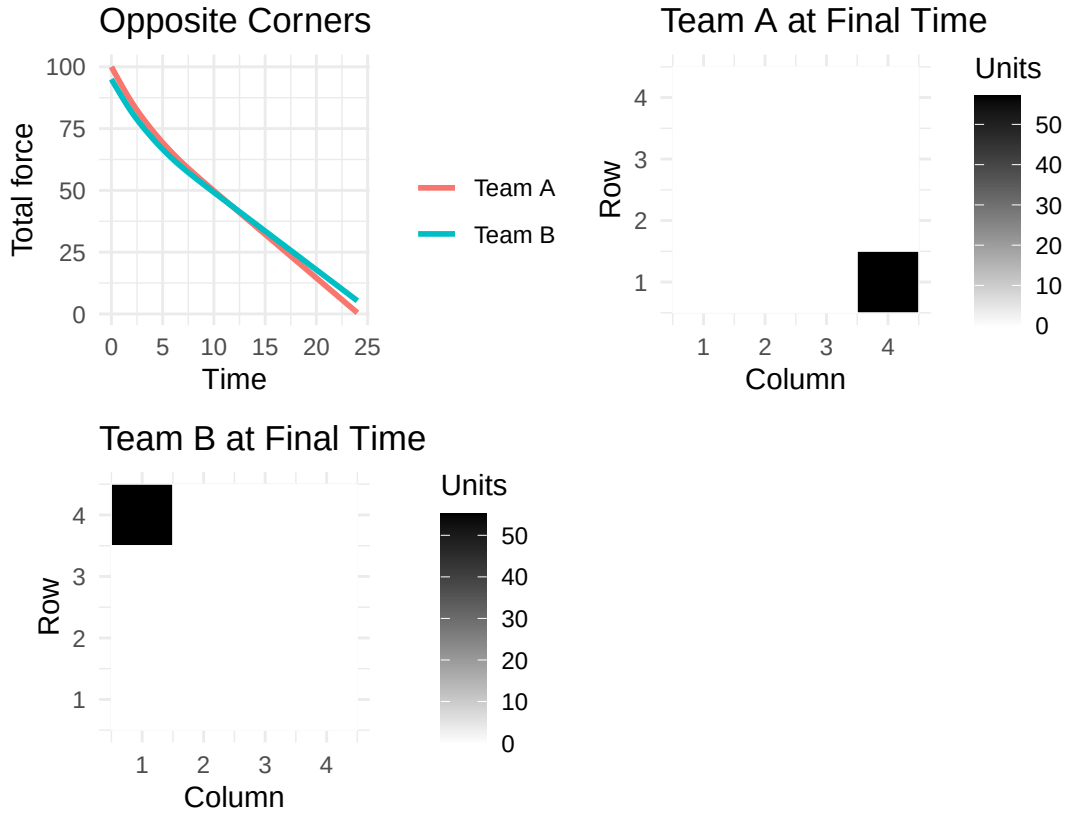
This is a good spatial analogue of a front-to-front battle.



In this scenario, the most intense fighting is expected in the central columns. If the fighting is strong enough, the side with the stronger effective reinforcement and combat rate should push the front line toward the enemy boundary.

## 4.2 Scenario II: Opposite Corners

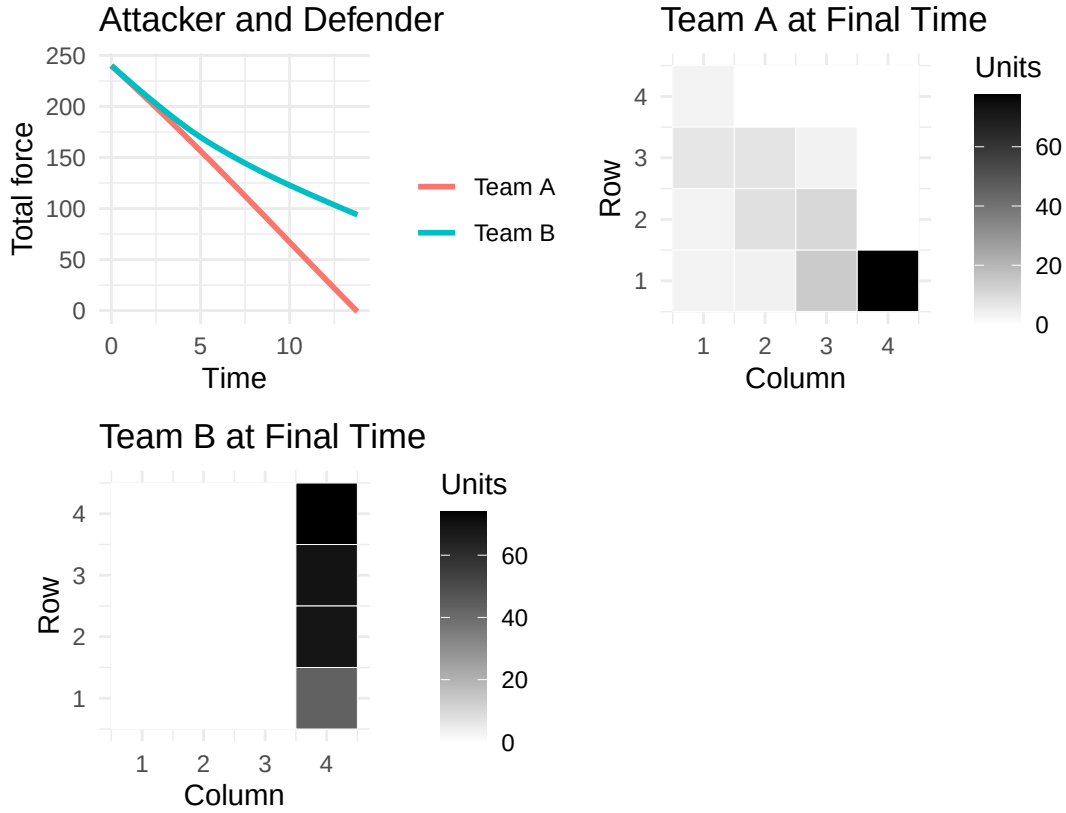
Now Team A starts at the top-left corner and moves southeast, while Team B starts at the bottom-right corner and moves northwest. This creates a diagonal approach toward the middle of the battlefield.



This case is useful because the main interaction occurs in the diagonal band of the grid rather than only along a single front.

## 4.3 Scenario III: One Attacker and One Defender

Here Team A attacks from the left boundary toward the right, while Team B mainly defends and remains in place. This scenario shows how a stronger mover may force contact even when the opponent is not advancing.

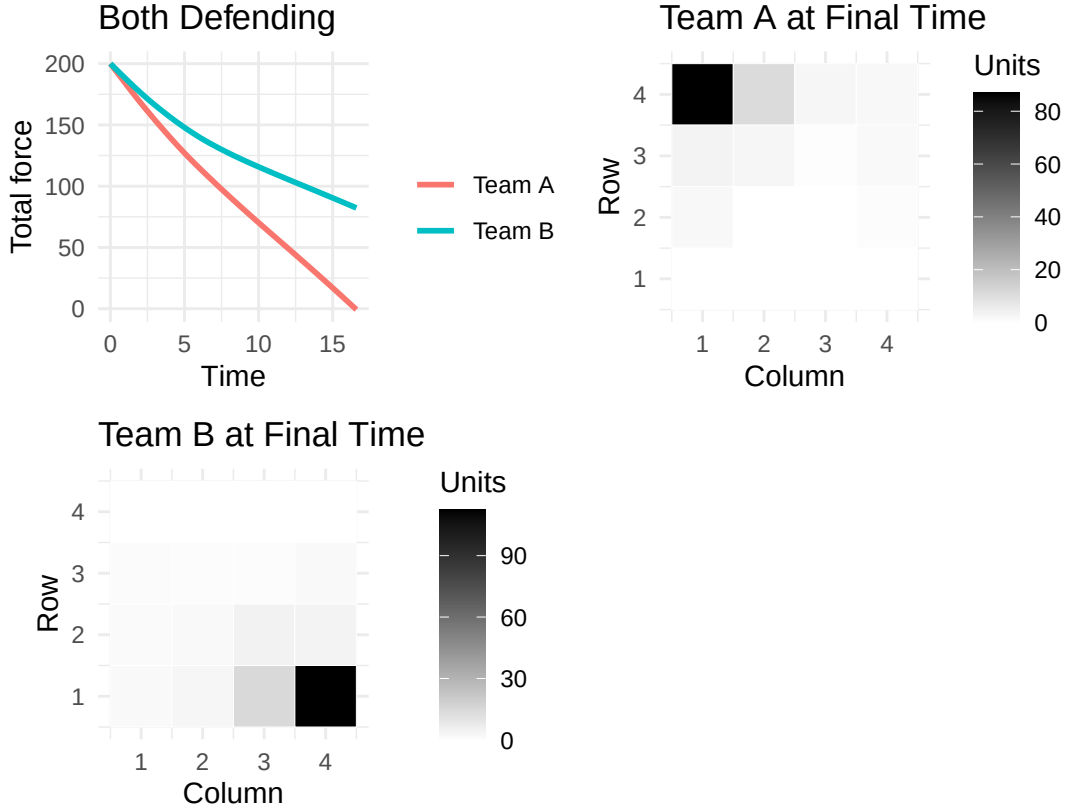


This case demonstrates a more asymmetric battlefield. Team A advances, while Team B primarily resists in place.

#### 4.4 Scenario IV: Both Teams Defending

In this final case, both teams move away from the center. Team A retreats leftward, while Team B retreats rightward. This is a useful stress test for the model because it may produce separation rather than sustained contact.





In this case, the two forces may drift apart, which can reduce sustained combat. This helps illustrate one limitation of naive movement rules: if the objectives of the two teams are too symmetric, the model may fail to generate a meaningful clash.

## 5 Winning Conditions

For the classical non-spatial model, a closed-form winning condition exists:

$$\alpha A_0^2 > \beta B_0^2 \Rightarrow \text{Team A wins,}$$

and

$$\beta B_0^2 > \alpha A_0^2 \Rightarrow \text{Team B wins.}$$

For the spatial model, exact closed-form winning conditions are usually much harder to derive because movement redistributes troops across the battlefield. However, some useful mathematical ideas remain.

### 5.1 Local winning condition

If one cell experiences a short, intense clash and movement is negligible inside that cell, then the local square law applies:

$$\alpha A_{ij}^2 - \beta B_{ij}^2 = C_{ij}$$

Thus, the side with larger local effective strength should dominate that cell.

## 5.2 Approximate global interpretation

If the battlefield is well mixed and the main combat takes place in a narrow contact zone, then the total force sizes can sometimes be interpreted through an effective square-law-like comparison of the contact band.

This does not give an exact theorem for the full spatial model, but it does provide a useful qualitative rule: the side with larger effective strength in the active front is more likely to advance.

## 5.3 Steady-state viewpoint

If reinforcements are present, then one may also study steady states by setting all derivatives to zero. In the discrete system, this gives a large algebraic system. Exact symbolic solutions are usually cumbersome, so numerical study is more practical.

# 6 Limitations and Further Directions

The present model is still simplified. Important extensions include:

- stochastic attrition, where losses happen randomly,
- heterogeneous troops, where not all units have the same strength,
- terrain effects, where some cells are easier to occupy than others,
- morale or fatigue, where effectiveness changes over time,
- delayed response, where troops do not react instantly,
- calibration using real or simulated data.

## Numerical Anomalies in the Heat-Map Plots

There is also an issue with the numerical solutions (mainly the 2D spatial *heat-map plots*) obtained from the codes. The issue is that even though all *total population vs. time* plots show that one of the teams get extinct eventually, but the *heat-map* plots show that some populations of both the teams remain at the final time. This could be due to few reasons: (i) there are still few numerical errors which are taken into account, (ii) somehow may be the heat-maps are plotting the population after getting some reinforcements right after the population of one team goes extinct, (iii) in the main codes which makes the population to forcefully go to 0, due to some bound on the  $\epsilon$ -errors when it is attained, (iv) the gradient scale shown in each heat-maps are not in the same scale (so 30 unit population of team A is not same a 30 unit population of team B), so it might be the case that population for the *close-to-extinct team* is getting *exaggerated* in terms of their population in these heat-maps.

*Model Improvement:* A natural next step is to combine several of these improvements into one generalized model.

For example, a more general version could be written as:

$$\frac{dA_{ij}}{dt} = -\beta_{ij}(t)B_{ij} + m_{ij}^A(t) + r_{ij}^A + \epsilon_{ij}^A,$$

$$\frac{dB_{ij}}{dt} = -\beta_{ij}(t)A_{ij} + m_{ij}^B(t) + r_{ij}^B + \epsilon_{ij}^B.$$

where,  $m_{ij}^k(t)$  depicts movement of team k at  $(i, j)$ ,

$r_{ij}^k$  shows the reinforcements for k at  $(i, j)$ ,

and,  $\epsilon_{ij}^k$  shows the random errors due to k at  $(i, j)$ .

## 7 Conclusion

This project began with the classical two-team Lanchester combat model and showed how its square-law structure leads to a clean winning condition. Because the classical model ignores space, it was then extended into a 2D spatial framework by discretizing the battlefield into cells and building a system of coupled ODEs. Several tactical situations were studied in R, including opposite-boundary attacks, opposite-corner engagement, attacker-defender interaction, and mutual defense. The project shows that the classical square law remains meaningful locally, but that spatial movement and reinforcements make the global dynamics more complex. This makes the spatial Lanchester model a strong basis for further research in stochastic, heterogeneous, and terrain-dependent extensions.

## References

- Lanchester-Type Models of Warfare. Volume I&II (DTIC)
- Nigel Perry, Fractal Effects in Lanchester Models of Combat, 2009, DEFENCE SCIENCE AND TECHNOLOGY ORGANISATION (AUSTRALIA) JOINT OPERATIONS DIVISION, DTIC
- N. Cangioti, M. Capolli, M. Sensi, A Generalization of Lanchester's Model of Warfare, 2022, Physics and Society; Dynamical Systems, Arxiv

# Appendices

## Annexure: All R Code

```
knitr::opts_chunk$set(echo = FALSE, message = FALSE, warning = FALSE)
library(deSolve)
library(ggplot2)
basic_lanchester <- function(t, state, parameters) {
  with(as.list(c(state, parameters)), {
    dA <- -beta * B
    dB <- -alpha * A
    list(c(dA, dB))
  })
}

# Parameters and initial conditions
par_basic <- c(alpha = 0.12, beta = 0.10)
state_basic <- c(A = 120, B = 100)
times_basic <- seq(0, 20, by = 0.1)

sol_basic <- ode(
  y = state_basic,
  times = times_basic,
  func = basic_lanchester,
  parms = par_basic
)

sol_basic <- as.data.frame(sol_basic)

# Stop solution when one force reaches zero
end_idx <- which(sol_basic$A <= 0 | sol_basic$B <= 0)[1]

if(!is.na(end_idx)) {
  sol_basic <- sol_basic[1:end_idx, ]
}

# Plot 1: Force sizes over time
p1 <- ggplot(sol_basic, aes(x = time)) +
  geom_line(aes(y = A, color = "Team A"), linewidth = 1) +
  geom_line(aes(y = B, color = "Team B"), linewidth = 1) +
  labs(
    x = "Time",
    y = "Force size",
    title = "Classical Lanchester Model",
    color = ""
  ) +
  theme_minimal()

# hyperbolic phase diagram wrt to teams
library(deSolve)
library(ggplot2)
```

```

basic_lanchester <- function(t, state, parameters) {
  with(as.list(c(state, parameters)), {
    dA <- -beta * B
    dB <- -alpha * A
    list(c(dA, dB))
  })
}

# Parameters and initial conditions
alpha <- 0.12
beta <- 0.10
A0 <- 120
B0 <- 100

par_basic <- c(alpha = alpha, beta = beta)
state_basic <- c(A = A0, B = B0)
times_basic <- seq(0, 20, by = 0.1)

sol_basic <- ode(
  y = state_basic,
  times = times_basic,
  func = basic_lanchester,
  parms = par_basic
)

sol_basic <- as.data.frame(sol_basic)

# Stop solution when one force reaches zero
end_idx <- which(sol_basic$A <= 0 | sol_basic$B <= 0)[1]

if(!is.na(end_idx)) {
  sol_basic <- sol_basic[1:end_idx, ]
}

# Invariant
C0 <- alpha * A0^2 - beta * B0^2

# Hyperbola branch in the first quadrant
A_grid <- seq(0, max(sol_basic$A) * 1.05, length.out = 500)
B_curve <- sqrt(pmax((alpha * A_grid^2 - C0) / beta, NA))

hyper_df <- data.frame(A = A_grid, B = B_curve)
traj_df <- data.frame(A = sol_basic$A, B = sol_basic$B)

# Plot 2: Hyperbolic phase-plane curve
p2 <- ggplot() +
  geom_path(data = traj_df, aes(x = A, y = B), linewidth = 1) +
  geom_line(data = hyper_df, aes(x = A, y = B), linetype = 2, linewidth = 1) +
  labs(
    x = "Team A population",
    y = "Team B population",
    title = "Phase-Plane Hyperbola"
  ) +

```

```

theme_minimal()

# Side-by-side display
library(patchwork)
p1 + p2 + plot_layout(ncol = 2)
# Convert between grid coordinates and vector index
cell_index <- function(i, j, n) {
  (i - 1) * n + j
}

# Build a movement map for one team
# direction can be: east, west, north, south, ne, nw, se, sw, stay
make_next_map <- function(n, direction) {

  grid <- expand_grid(i = 1:n, j = 1:n)
  next_map <- integer(n * n)

  for (s in seq_len(n * n)) {

    i <- grid$i[s]
    j <- grid$j[s]

    if (direction == "east") {

      ii <- i
      jj <- min(j + 1, n)

    } else if (direction == "west") {

      ii <- i
      jj <- max(j - 1, 1)

    } else if (direction == "north") {

      ii <- max(i - 1, 1)
      jj <- j

    } else if (direction == "south") {

      ii <- min(i + 1, n)
      jj <- j

    } else if (direction == "ne") {

      ii <- max(i - 1, 1)
      jj <- min(j + 1, n)

    } else if (direction == "nw") {

      ii <- max(i - 1, 1)
      jj <- max(j - 1, 1)

    } else if (direction == "se") {

```

```

    ii <- min(i + 1, n)
    jj <- min(j + 1, n)

  } else if (direction == "sw") {

    ii <- min(i + 1, n)
    jj <- max(j - 1, 1)

  } else if (direction == "stay") {

    ii <- i
    jj <- j

  } else {

    stop("Unknown direction.")

  }

  next_map[s] <- cell_index(ii, jj, n)
}

next_map
}

# Create a sparse source vector for reinforcements
make_source <- function(n, cells, rate) {

  v <- rep(0, n * n)
  v[cells] <- rate

  v
}

# Make a spatial RHS function for deSolve
make_spatial_rhs <- function(
  n,
  nextA,
  nextB,
  alpha,
  beta,
  kA,
  kB,
  srcA,
  srcB
) {

  ncell <- n * n

  predA <- lapply(
    seq_len(ncell),
    function(s) which(nextA == s & seq_len(ncell) != s)

```

```

)

predB <- lapply(
  seq_len(ncell),
  function(s) which(nextB == s & seq_len(ncell) != s)
)

outA <- as.integer(nextA != seq_len(ncell))
outB <- as.integer(nextB != seq_len(ncell))

function(t, state, parameters) {

  A <- pmax(state[1:ncell], 0)

  B <- pmax(
    state[(ncell + 1):(2 * ncell)],
    0
  )

  dA <- numeric(ncell)
  dB <- numeric(ncell)

  for (s in seq_len(ncell)) {

    inflowA <- kA * sum(A[predA[[s]]])
    outflowA <- kA * outA[s] * A[s]

    inflowB <- kB * sum(B[predB[[s]]])
    outflowB <- kB * outB[s] * B[s]

    dA[s] <- inflowA -
      outflowA -
      beta * B[s] +
      srcA[s]

    dB[s] <- inflowB -
      outflowB -
      alpha * A[s] +
      srcB[s]

  }

  list(c(dA, dB))

}
}

# Utility for plotting a snapshot as a heatmap
plot_snapshot <- function(
  sol,
  n,
  team = c("A", "B"),
  time_point = NULL,

```



```

title = NULL,
fill_max = NULL
) {

  team <- match.arg(team)

  ncell <- n * n

  if (is.null(time_point)) {

    row_id <- nrow(sol)

  } else {

    row_id <- which.min(abs(sol[, 1] - time_point))

  }

  if (team == "A") {

    vals <- sol[row_id, 2:(ncell + 1)]

  } else {

    vals <- sol[row_id, (ncell + 2):(2 * ncell + 1)]

  }

  df <- expand.grid(i = 1:n, j = 1:n)

  # Remove tiny numerical negatives
  df$value <- ifelse(
    as.numeric(vals) < 1e-3,
    0,
    as.numeric(vals)
  )

  if (is.null(title)) {

    title <- paste("Team", team, "- Spatial Snapshot")

  }

  # Common scale if not supplied
  if (is.null(fill_max)) {

    fill_max <- max(df$value)

  }

  ggplot(df, aes(x = j, y = n - i + 1, fill = value)) +

    geom_tile(color = "white") +

```

```

coord_equal() +

scale_fill_gradient(
  low = "white",
  high = "black",
  limits = c(0, fill_max)
) +

labs(
  x = "Column",
  y = "Row",
  fill = "Units",
  title = title
) +

theme_minimal()
}

# Utility for plotting total forces over time
plot_totals <- function(
  sol,
  n,
  title = "Total Forces Over Time"
) {

  ncell <- n * n

  out <- as.data.frame(sol)

  totalA <- rowSums(
    out[, 2:(ncell + 1)]
  )

  totalB <- rowSums(
    out[, (ncell + 2):(2 * ncell + 1)]
  )

  df <- data.frame(
    time = out$time,
    TeamA = totalA,
    TeamB = totalB
  )

  df_long <- rbind(
    data.frame(
      time = df$time,
      team = "Team A",
      value = df$TeamA
    ),
    data.frame(
      time = df$time,
      team = "Team B",
      value = df$TeamB
    )
  )
}

```

```

    )
  )

  ggplot(
    df_long,
    aes(x = time, y = value, color = team)
  ) +

    geom_line(linewidth = 1) +

    labs(
      x = "Time",
      y = "Total force",
      title = title,
      color = ""
    ) +

    theme_minimal()
}

# Stop solution once one team becomes effectively extinct
truncate_solution <- function(
  sol,
  n,
  eps = 1
) {

  ncell <- n * n

  out <- as.data.frame(sol)

  totalA <- rowSums(
    out[, 2:(ncell + 1)]
  )

  totalB <- rowSums(
    out[, (ncell + 2):(2 * ncell + 1)]
  )

  end_idx <- which(
    totalA <= eps | totalB <= eps
  )[1]

  if (!is.na(end_idx)) {

    out <- out[1:end_idx, ]

  }

  out
}

n <- 4
ncell <- n * n

```

```

grid <- expand.grid(i = 1:n, j = 1:n)

# Initial conditions: troops start on opposite boundaries
A0 <- rep(0, ncell)
B0 <- rep(0, ncell)

left_cells <- which(grid$j == 1)
right_cells <- which(grid$j == n)

A0[left_cells] <- 60
B0[right_cells] <- 55

# Reinforcement sources at the entry boundaries
srcA1 <- make_source(n, left_cells, rate = 2)
srcB1 <- make_source(n, right_cells, rate = 2)

# Movement directions
nextA1 <- make_next_map(n, "east")
nextB1 <- make_next_map(n, "west")

rhs1 <- make_spatial_rhs(
  n = n,
  nextA = nextA1,
  nextB = nextB1,
  alpha = 0.08,
  beta = 0.10,
  kA = 0.60,
  kB = 0.60,
  srcA = srcA1,
  srcB = srcB1
)

times1 <- seq(0, 30, by = 0.2)
sol1 <- ode(
  y = c(A0, B0),
  times = times1,
  func = rhs1,
  parms = NULL
)

sol1 <- truncate_solution(sol1, n)

p1 <- plot_totals(sol1, n, title = "Opposite Boundaries")
p2 <- plot_snapshot(sol1, n, "A", title = "Team A at Final Time")
p3 <- plot_snapshot(sol1, n, "B", title = "Team B at Final Time")

library(patchwork)
p1 + p2 + p3 + plot_layout(ncol = 2)

A0 <- rep(0, ncell)
B0 <- rep(0, ncell)

A0[cell_index(1, 1, n)] <- 100

```

```

B0[cell_index(n, n, n)] <- 95

srcA2 <- make_source(n, c(cell_index(1, 1, n)), rate = 2)
srcB2 <- make_source(n, c(cell_index(n, n, n)), rate = 2)

nextA2 <- make_next_map(n, "se")
nextB2 <- make_next_map(n, "nw")

rhs2 <- make_spatial_rhs(
  n = n,
  nextA = nextA2,
  nextB = nextB2,
  alpha = 0.09,
  beta = 0.10,
  kA = 0.55,
  kB = 0.55,
  srcA = srcA2,
  srcB = srcB2
)

times2 <- seq(0, 35, by = 0.2)
sol2 <- ode(
  y = c(A0, B0),
  times = times2,
  func = rhs2,
  parms = NULL
)

sol2 <- truncate_solution(sol2, n)

p1 <- plot_totals(sol2, n, title = "Opposite Corners")
p2 <- plot_snapshot(sol2, n, "A", title = "Team A at Final Time")
p3 <- plot_snapshot(sol2, n, "B", title = "Team B at Final Time")

library(patchwork)
p1 + p2 + p3 + plot_layout(ncol = 2)
A0 <- rep(0, ncell)
B0 <- rep(0, ncell)

A0[left_cells] <- 60
B0[right_cells] <- 60

srcA3 <- make_source(n, left_cells, rate = 2)
srcB3 <- make_source(n, right_cells, rate = 1)

nextA3 <- make_next_map(n, "east")
nextB3 <- make_next_map(n, "stay")

rhs3 <- make_spatial_rhs(
  n = n,
  nextA = nextA3,
  nextB = nextB3,
  alpha = 0.08,

```

```

    beta = 0.10,
    kA = 0.60,
    kB = 0.00,
    srcA = srcA3,
    srcB = srcB3
  )

times3 <- seq(0, 30, by = 0.2)
sol3 <- ode(
  y = c(A0, B0),
  times = times3,
  func = rhs3,
  parms = NULL
)

sol3 <- truncate_solution(sol3, n)

p1 <- plot_totals(sol3, n, title = "Attacker and Defender")
p2 <- plot_snapshot(sol3, n, "A", title = "Team A at Final Time")
p3 <- plot_snapshot(sol3, n, "B", title = "Team B at Final Time")

library(patchwork)
p1 + p2 + p3 + plot_layout(ncol = 2)
A0 <- rep(0, ncell)
B0 <- rep(0, ncell)

A0[right_cells] <- 50
B0[left_cells] <- 50

srcA4 <- make_source(n, right_cells, rate = 1)
srcB4 <- make_source(n, left_cells, rate = 1)

nextA4 <- make_next_map(n, "west")
nextB4 <- make_next_map(n, "east")

rhs4 <- make_spatial_rhs(
  n = n,
  nextA = nextA4,
  nextB = nextB4,
  alpha = 0.08,
  beta = 0.10,
  kA = 0.45,
  kB = 0.45,
  srcA = srcA4,
  srcB = srcB4
)

times4 <- seq(0, 30, by = 0.2)
sol4 <- ode(
  y = c(A0, B0),
  times = times4,
  func = rhs4,
  parms = NULL

```

```
)  
  
sol4 <- truncate_solution(sol4, n)  
  
p1 <- plot_totals(sol4, n, title = "Both Defending")  
p2 <- plot_snapshot(sol4, n, "A", title = "Team A at Final Time")  
p3 <- plot_snapshot(sol4, n, "B", title = "Team B at Final Time")  
  
library(patchwork)  
p1 + p2 + p3 + plot_layout(ncol = 2)
```